

**1. Adott egy 10 elemű sorozat, a sorozat elemeit tároljuk egy tömbben.
Rendezzük az A tömböt közvetlen kiválasztással, minimum kiválasztással,
buborék rendezéssel és beszűrő rendezéssel is!**

```
#include <iostream>
using namespace std;
int main()
{
    int seged,ertek,i,j;
    int A[10]={13,28,43,37,10,5,8,99,101,25};

    // rendezetlen tömb kiíratása
    for (int i=0;i<10;i++)
        cout<<A[i]<<" ";
    /*****rendezés közvetlen kiválasztással
    for(int i=0;i<9;i++)
    {
        for(int j=i+1;j<10;j++)
        {
            if(A[i]>A[j])
            {
                seged=A[i];
                A[i]=A[j];
                A[j]=seged;
            }
        }
    }*/
    /*****rendezés minimum kiválasztással
    for(i=0;i<9;i++)
    {
        seged=i;
        ertek=A[i];
        for(j=i+1;j<10;j++)
        {
            if(ertek>A[j])
            {
                ertek=A[j];
                seged=j;
            }
        }
        A[seged]=A[i];
        A[i]=ertek;
    }*/
    /*****buborékos rendezés
    for(i=1;i<10;i++)
    {
        for(j=9;j>=i;j--)
        {
            if(A[j]<A[j-1])
            {
                seged=A[j-1];
```

```

        A[j-1]=A[j];
        A[j]=seged;
    }
}
}*/
/*****
egyszerű beillesztéses rendezés*/
for(j=1;j<10;j++)
{
    i=j-1;
    seged=A[j];
    while(i>=0 && seged<A[i])
    {
        A[i+1]=A[i];
        i--;
    }
    A[i+1]=seged;
}
// rendezett tömb kiírása
cout<<endl;
for(int i=0;i<10;i++)
    cout<<A[i]<<" ";
return 0;
}

```

2. Egy utazó ügynök 6 várost látogatott meg a héten. A meglátogatott városok neveit egy tömbben tárolta. Rendezzük a tömbben lévő 6 város nevét abc sorrendbe!

```

#include <iostream>
#include <string>
using namespace std;
int main()
{
    // Adatbevitel
    int i=0,j;
    string tmb[6]={ "Cegled", "Pecs", "Vac", "Miskolc", "Ada", "Kaposvar" };
    // rendezés előtt
    cout<<"Rendezes előtt :"<<endl;
    for (int i = 0; i < 6; i++)
    {
        cout<< tmb[i]<<" ";
    }
    //RENDEZÉS
    string seged;
    for(int i=0;i<5;i++)
    {
        for(int j=i+1;j<6;j++)
        {
            if(tmb[i]>tmb[j])

```

```

        {
            seged=tmb[i];
            tmb[i]=tmb[j];
            tmb[j]=seged;
        }
    }
}
//Rendezett tömb kiírása a képernyőre
cout << "\nRendezes utan :" << endl;
for(i=0; i<6;i++)
{
    cout<<tmb[i]<<" ";
}
cout << endl;
return 0;
}
/*

```

- 3. Egy 16 csapatos labdarúgó bajnokság egyik fordulójának eredményeit tároljuk a merkozes.txt állományban. Ebben a fordulóban nem biztos, hogy 8 mérkőzés volt, néhány meccs elmaradt a rossz időjárás miatt. Minden mérkőzés eredménye új sorban van és a gólok, valamint a csapatok száma szóközzel elválasztva. (3 2 ute vac) Olvassa be a fájlt egy olyan adatszerkezetbe, amit az alábbi kérdésekhez fel tud használni (struktúra)) és írassa ki hány mérkőzés volt a fordulóban! Rendezze a csapatokat névsor szerint növekvő sorrendben és írja ki a csapatok txt állományba**

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;
struct fordulo
{
    int lott, kapott ;
    string nev1, nev2;
};
int main()
{
    // Adatbevitel

    ifstream be("merkozes.txt");
    if (be.fail()) { cerr << "hiba"; system("pause"); exit(1); }
    int db = 0;
    fordulo A[8];
    while(be >> A[db].lott>> A[db].kapott>>A[db].nev1>>A[db].nev2)
    {
        cout << A[db].lott << " " << A[db].kapott << " " << A[db].nev1 << " " << A[db].nev2
<< endl;
        db++;
    }
}

```

```

cout << "\n A forduloban :." << db << " merkozest játszottak" << endl;
be.close();
cout << endl;
//Rendezze acsapatokat névsor szerint növekvő sorrendben
//és írja ki a csapatok.txt állományba
string seged[16];
string temp;
int j = 0;
for (int i = 0; i < db; i++)
{
    seged[j] = A[i].nev1;
    j++;
    seged[j] = A[i].nev2;
    j++;
}
ofstream ki("csapatok.txt");
if (ki.fail()) { cerr << "hiba"; system("pause"); exit(1); }

for (int i = 0; i < 2 * db - 1; i++)
{
    for (j = i + 1; j < 2 * db; j++)
    {
        if (seged[i] > seged[j])
        {
            temp = seged[i];
            seged[i] = seged[j];
            seged[j] = temp;
        }
    }
}
for (int i = 0; i < 2 * db; i++)
{
    cout << seged[i] << endl;
    ki << seged[i] << endl;
}
ki.close();
return 0;
}

```

4. Egy 16 csapatos labdarúgó bajnokság egyik fordulójának eredményeit tároljuk a merkozes.txt állományban. Ebben a fordulóban nem biztos, hogy 8 mérkőzés volt, néhány meccs elmaradt a rossz időjárás miatt. Minden mérkőzés eredménye új sorban van és a gólok, valamint a csapatok száma szóközzel elválasztva. (3 2 ute vac)

Olvassa be a fájlt egy olyan adatszerkezetbe, amit az alábbi kérdésekhez fel tud használni (struktúra)) és írassa ki hány mérkőzés volt a fordulóban!

- Mennyi csapat nyert otthon.
- Melyik csapat rúgta a legtöbb gólt.
- Volt -e döntetlen.
- Mennyi gólt lőtt az UTE (tudjuk, hogy játszott).
- Rendezze a csapatokat névsor szerint növekvő sorrendbe és írja ki a csapatok.txt állományba

Az egyes feladatrészeket függvényekkel oldja meg!

```
#include <iostream>/
#include <fstream>
#include <string>
using namespace std;
struct fordulo
{
    int lott, kapott ;
    string nev1, nev2;
};
int adatbe(fordulo * tmb);
void rendez(fordulo *tmb, int d);

int main()
{
    fordulo A[8];
    int i=0,j,db=adatbe(A);
    cout<<"\n A forduloban :"<<db<<" merkozest jatszottak"<<endl;
    cout<<"A csapatok nevsora rendezve :"<<endl;
    rendez(A,db);
    return 0;
}

int adatbe(fordulo * tmb)
{
    int db=0;
    ifstream be("merkozes.txt");
    if(be.fail()){cout<<"hiba";system("pause");exit(1);}
    while(be >> tmb[db].lott >> tmb[db].kapott>> tmb[db].nev1>> tmb[db].nev2)
    {
        cout<<tmb[db].lott<<" "<<tmb[db].kapott<<" "<<tmb[db].nev1<<"
"<<tmb[db].nev2<<endl;
        db++;
    }
    be.close();
    return db;
}
```

```

}
void rendez(fordulo *tmb, int d)
{
    string seged[16];
    string temp;
    int i, j=0;
    for(i=0;i<d;i++)
    {
        seged[j]=tmb[i].nev1;
        j++;
        seged[j]=tmb[i].nev2;
        j++;
    }
    ofstream ki("csapatok.txt");
    if(ki.fail()){cout<<"hiba";system("pause");exit(1);}

    for(i=0;i<2*d-1;i++)
    {
        for(j=i+1;j<2*d;j++)
        {
            if(seged[i]>seged[j])
            {
                temp=seged[i];
                seged[i]=seged[j];
                seged[j]=temp;
            }
        }
    }

    for(i=0;i<2*d;i++)
    {
        cout<<seged[i]<<endl;
        ki<<seged[i]<<endl;
    }
    ki.close();
}

```

5. Egy utazó ügynök a vartav.txt állományban tárolja a januárban felkeresett városok nevét és a megtett napi kilométereket. A hónapban nem volt 20 városnál többen s a városok nevei szóközzel elválasztva vannak a távolságtól. Az egyes városok mindig új sorban vannak. Készítsünk programot mely kimutatja:
- Mennyi kilométert utazott januárban.
 - Melyik a legtávolabbi város.
 - Mennyiszer volt közeli városban (távolság <80km)
 - Rendezze az adatokat növekvő sorrendben távolság szerint, írassa ki képernyőre majd a rendezettavolsag.txt állományba.

Az egyes feladatokat függvényekkel oldjuk meg!

```
#include <iostream>
#include <fstream>
#include <string>
using namespace std;
struct ugynok
{
    string varos;
    int tav;
};
int beolvas(char * t, ugynok *tmb);
int ossztav(ugynok * tmb, int db);
string legtavolabb(ugynok * tmb, int db);
int kozeliekszama(ugynok * tmb, int db);
void rendezvekiir(ugynok * tmb, int db);
int main()
{
    char megnyit[]="vartav.txt";
    ugynok A[20];
    int varosdb=beolvas(menyit, A);
    cout<<endl<< "A meglátogatott városok száma"<<varosdb<<endl;
    cout<<"A januárban megtett össz. távolság " <<ossztav(A,varosdb)<<endl;
    cout<<"A legtávolabbi város neve: " <<legtavolabb(A, varosdb)<<endl;
    cout<<" 80 km-nel közelebbi városok száma " <<kozeliekszama(A,
varosdb)<<endl;
    rendezvekiir(A,varosdb);
    system("pause");
    return 0;
}
int beolvas(char * t, ugynok *tmb)
{
    ifstream be(t);
    if(be.fail()){cout<<"hiba";system("pause");exit(1);}
    int i,j,db=0;
    while(be>>tmb[db].varos>>tmb[db].tav)
    {
        cout<<tmb[db].varos<<" " <<tmb[db].tav<<endl;
        db++;
    }
}
```

```

        be.close();
    return db;
}
int ossztav(ugynok * tmb, int db)
{
    int ossz=0;
    for(int i=0; i<db;i++)
        ossz=ossz+tmb[i].tav;
    return 2*ossz;
}
string legtavolabb(ugynok * tmb, int db)
{
    int max=0;
    for(int i=1; i<db;i++)
    {
        if(tmb[i].tav>tmb[max].tav){ max=i;}
    }
    return tmb[max].varos;
}
int kozeliekszama(ugynok * tmb, int db)
{
    int tav=0;
    for(int i=0; i<db; i++)
        if(tmb[i].tav<80) tav++;
    return tav;
}
void rendezvekiir(ugynok * tmb, int db)
{
    ugynok seged;
    ofstream ki("rendezetttavolsag.txt");
    if(ki.fail()){cerr<<"hiba";system("pause");exit(1);}
    for(int i=0;i<db-1;i++)
    {
        for(int j=i+1;j<db;j++)
        {
            if(tmb[i].tav>tmb[j].tav)
            {
                seged=tmb[i];
                tmb[i]=tmb[j];
                tmb[j]=seged;
            }
        }
    }
    for(int i=0;i<db;i++)
    {
        cout<<tmb[i].varos<<" "<<tmb[i].tav<<endl;
        ki<<tmb[i].varos<<" "<<tmb[i].tav<<endl;
    }
    ki.close();
}

```